FC4SC - A User's Guide

Jens Schönherr
HTW Dresden
jens.schoenherr@htw-dresden.de

October 27, 2026

1 What is FC4SC?

FC4SC is a C++-library to measure the functional coverage during a simulation. It has been developed by AMIQ Consulting [AMI18] and is now maintained by the Accellera SystemC Verification Working Group [Acc26a].

There are no officially numbered releases of FC4SC. The git repository [Acc19] provides the recent version. This document refers to commit 31b9339.

2 Installation

There is no installation needed. FC4SC is a collection of C++-template classes that are contained in a set of header files. To use FC4SC, include `fc4sc.hpp` and set the include path of the compiler to the `include` directory of FC4SC.

```
#include "fc4sc.hpp"
```

Listing 2.1: Inclusion of main header file of FC4SC

FC4SC requires C++11. With some minor changes it can be compiled with C++17:

- In `include/fc4sc_base.hpp` the declarations `class cvg_metadata;` and `class scp_metadata;` must be changed to `struct cvg_metadata;` and `struct scp_metadata;` to fit to its definitions.
- In `include/fc4sc_coverpoint.hpp` the `operator()` of `struct IntervalComp` must be declared as `const`.

FC4SC uses the type `uint`. If this type is not defined in the standard header files it should be defined by `typedef unsigned int uint;` before including the FC4SC header files.

The patches provides by [Sch26] fix these issues and substitute the usage of `uint` by appropriate types of `cstdint`. In Section 6 these substitutions are marked.

To learn more about the internal structure of FC4SC a documentation can be created by doxygen. This User's guide focuses on the *usage* of FC4FC, only, since the user documentation provided in [Acc19] is incomplete and refers to the AMIQ version [AMI18].

```
cd doc
doxygen Doxyfile
```

Listing 2.2: Create FC4SC internal documentation

3 Tutorial

Before going into the details in Section 4 this tutorial section shows the basic ideas behind FC4SC and allows to create a first running example.

Functional coverage is a method that measures if and how often certain situations occur during a simulation. FC4SC is aimed to the simulation of circuits using SystemC (**F**unctional **C**overage for **S**ystem**C**). However, there is no dependency to SystemC.

A situation in a simulation is defined by variables and their values. In FC4SC, variables that are to be evaluated are attributes of a covergroup class. Covergroup objects are typically contained in the scoreboard of a simulation. Listing 3.1 shows the declaration of a covergroup class `my_covergroup` and its instantiation inside a class `sb` (scoreboard) or as local variable in a function `f()`.

The covergroup class `my_covergroup` needs some basic definition that are provided by the macro `CG_CONS` with the name of the covergroup class as parameter.

```
class my_covergroup : public fc4sc::covergroup
{
public:
  CG_CONS(my_covergroup) {}

  int var1;
  int var2;

  COVERPOINT(int, cp1,  var1, true)
  {
    bin<int>("zero", 0),
    bin<int>("one",  1),
    bin<int>("two",  2)
  };

  COVERPOINT(int, cp2,  var2, true)
  {
    bin<int>("zero",  0),
    bin<int>("pos1", +1),
    bin<int>("neg1", -1)
  };

  cross<int, int> cp1_cp2_cr =
  cross<int, int>(this, "cp1_cp2_cr", &cp1, &cp2);
};

// covergroup as member variable (attribute)
class sb
{
public:
  my_covergroup cg1;
  // constructor of sb calls constructor of covergroup
  sb() : cg1("cg1") {}
};

// covergroup as local variable
void f()
{
  my_covergroup cg2("cg2");
  cg2.var1 = 0;
  cg2.var2 = 1;
  cg2.sample();
  // ...
  cg2.stop();

  xml_printer::coverage_save("coverage.xml");
}
```

Listing 3.1: Example covergroup class `my_covergroup` and covergroup objects `cg1` and `cg2`

The class `my_covergroup` contains two attributes `var1` and `var2`. The values of these variables are evaluated in this class by calling the member function `sample()`.

It must be defined which values of those variables are interesting and should be covered during a simulation. This definition is achived by coverpoints. A coverpoint is a member of the covergroup that has a name, a data type, and a set of bins. In Listing 3.1 the coverpoint `cp1` evaluates `int` values of `var1`. For the evaluation it has three bins for the values 0, 1, and 2. The bins have the same type as the coverpoint. If `var1 = 0` when function `sample()` is called then bin `"zero"` has been hit and the hit counter of bin `"zero"` is incremented. If `var1 = 1` is sampled then bin `"one"` has been hit etc. If `var1` has a value for which no bin exists then no hit counter is incremented. The coverpoint `cp2` samples in a similar way values of `var2`.

FC4SC allows the evaluation of the functional coverage also for combinations of values. These combinations are called “crosses”. Crosses are defined by two or more coverpoints. In the definition of a cross, the data types of the coverpoints and references to the coverpoints must be specified together with the name of the cross. The

cross `cp1_cp2_cr` of Listing 3.1 contains implicitely $3 \cdot 3 = 9$ bins - each combination of the bins of `cp1` and `cp2` from ("zero", "zero") to ("two", "neg1").

Table 3.1: The 9 bins of cross `cp1_cp2_cr`

cp2 \ cp1:	zero	one	two
zero			
pos1			
neg1			

At the end of a simulation the sampling is stopped (by calling `stop()`) and the achieved coverage is saved in an UCIS file [Acc26b] i.e. a standardized XML format for coverage data.

4 Describing functional coverage

4.1 Contexts

FC4SC is made for measuring the functional coverage of a simulation. The data for this measurement is maintained in contexts. A context must explicitely be created and deleted (Listing 4.1).

```
void f()
{
  fc4sc::global* cntxt = fc4sc::global::create_new_context();

  fc4sc::global::delete_context(cntxt);
}
```

Listing 4.1: Creating and deleting a context

There is an default global context available that needs not to be created and deleted explicitly. A pointer to this context is returned by `fc4sc::global::getter()`. It is not allowed to delete the global context.

```
void f()
{
  fc4sc::global* glob_cntxt = fc4sc::global::getter();
}
```

Listing 4.2: Accessing the global context

The data in a context is completely separated from the data in all other contexts. It is not supported to combine data from different contexts.

4.2 Scopes

4.2.1 Top-level scopes

In complex simulation, the functional coverage is measured in many different places. To structure complex coverage measurements8, the different parts of the coverage measurement (covergroups, see Section 4.3) may belong to different scopes. Conceptually, a context contains one or more scope types each of which has several (scope) instances. A scope type is defined by a C++-class that is derived from class `fc4sc::scope` (Listing 4.3). Each scope class needs some declarations and definitions that are derived from the name of the scope class by the macro `SCOPE_DECL`. In addition, the scope class needs a constructor. This constructor gets at least the name

of the scope instance (i.e. object), the location of the scope instance (source code file name and line number), and the context, the scope instance belongs to. The constructor of a scope class calls the constructor of the base class `fc4sc::scope` and hands over the name of the scope instance, the location of the scope instance, and the context. Moreover, the `fc4sc::scope` constructor gets the location of the scope constructor. The constructor may have additional parameters and may initialize additional attributes of the scope class.

```
class my_scope : public fc4sc::scope
{

public:
  SCOPE_DECL(my_scope)

  my_scope(std::string name_,
           const char* auto_inst_file_name = "", int auto_line = 1, // loc. of the scope inst.
           fc4sc::global* cntxt = fc4sc::global::getter()) :
    fc4sc::scope(name_,
                 __FILE__, __LINE__,             // loc. of the scope type/class (my_scope)
                 auto_inst_file_name, auto_line, // loc. of the scope instance/object
                 cntxt)
    { } // further initialisations

};
```

Listing 4.3: Declaration of a scope class

At the instantiation of a scope class the name of the instance is handed over to the constructor as well as the location of the instantiation in the source code and the context. It is up to the user to ensure that all instances of a scope class have different names in a context. A synonym for "scope instance" is "module".

```
void f()
{
  my_scope my_scope_inst("my_scope_inst", __FILE__, __LINE__, cntxt);

  my_scope my_scope_global("my_scope_global", __FILE__, __LINE__); // global context
}
```

Listing 4.4: Instantiation of a scope class

In each context exists a scope "`default_scope`" with an instance "`default_scope_instance`". The default scope of a context can be obtained by `fc4sc::global::get_default_scope(cntxt)`. The parameter `cntxt` has a default value of the global context. The return value is a `scp_base*` i.e. a pointer to an instance of class `default_scope`. `scp_base` is the base class of class `scope` and class `default_scope`. They provide the same member functions (see Section 8.1).

Please note that `default_scope` and `default_scope_instance` are created when needed. If only self-defined scopes are used then `default_scope` and `default_scope_instance` are not available.

4.2.2 Sub-scopes

For a better structure of complex scopes, it is possible to define sub scopes as part of scopes. Sub scopes are sometimes called childs of their parent scopes.

A context may have several (≥ 1) top-level scope types i.e. scope types that do not have a parent scope type. There may be several (≥ 0) sub scope types of each scope type. Each sub scope type has exactly one parent scope type. The parent scope type is specified as a second parameter of macro `SCOPE_DECL()`.

Each scope type has several (≥ 0) scope instances (objects).

```
class top_scope : public fc4sc::scope
{
public:
  SCOPE_DECL(top_scope) // top-level scope type

  top_scope(std::string name_,
            const char* auto_inst_file_name = "", int auto_line = 1,
            fc4sc::global* cntxt = fc4sc::global::getter()) :
    fc4sc::scope(name_, __FILE__, __LINE__, auto_inst_file_name, auto_line, cntxt)
  { }
};

class sub1_scope : public fc4sc::scope
{
public:
  SCOPE_DECL(sub1_scope, top_scope)  // top_scope is parent scope type

  sub1_scope(fc4sc::scp_base& p_mod, // instance of top_scope
             std::string name_,
             const char* auto_inst_file_name = "", int auto_line = 1) :
    fc4sc::scope(p_mod, name_, __FILE__, __LINE__, auto_inst_file_name, auto_line)
  { }
};

class sub2_scope : public fc4sc::scope
{
public:
  SCOPE_DECL(sub2_scope, top_scope)  // top_scope is parent scope type

  sub2_scope(fc4sc::scp_base& p_mod, // instance of top_scope
             std::string name_,
             const char* auto_inst_file_name = "", int auto_line = 1) :
    fc4sc::scope(p_mod, name_, __FILE__, __LINE__, auto_inst_file_name, auto_line)
  { }
};

class subsub_scope : public fc4sc::scope
{
public:
  SCOPE_DECL(subsub_scope, sub1_scope) // sub1_scope is parent scope type

  subsub_scope(fc4sc::scp_base& p_mod, // instance of sub1_scope
               std::string name_,
               const char* auto_inst_file_name = "", int auto_line = 1) :
    fc4sc::scope(p_mod, name_, __FILE__, __LINE__, auto_inst_file_name, auto_line)
  { }
};
```

Listing 4.5: Hierarchy of scope types/classes

The name of a top-level scope type is the name of the class (ensured by `SCOPE_DECL(top_scope)`). The name of a sub scope type of another scope type is the name of the type of the parent scope type extended by `::` and the name of the sub scope type. The name of a scope type is provided by the static member function `scp_type_name()` of `scope` classes.

```
void f()
{
  std::string s1 = top_scope::scp_type_name();     // "top_scope"
  std::string s2 = sub1_scope::scp_type_name();    // "top_scope::sub1_scope"
  std::string s3 = sub2_scope::scp_type_name();    // "top_scope::sub2_scope"
  std::string s4 = subsub_scope::scp_type_name(); // "top_scope::sub1_scope::subsub_scope"
}
```

Listing 4.6: Names of of scope types of Listing 4.5

When a sub scope is instantiated a reference to a parent scope instance must be specified. The name of a scope instance is just the name specified as parameter of the constructor and, hence, does not contain any hierarchy information.

```
void f()
{
  fc4sc::global* cntxt = fc4sc::global::create_new_context();
  top_scope    top_scp1(               "top_scp1",    __FILE__, __LINE__, cntxt);
  sub1_scope   sub1_scp1(  top_scp1,  "sub1_scp1",   __FILE__, __LINE__);
  sub2_scope   sub2_scp1(  top_scp1,  "sub2_scp1",   __FILE__, __LINE__);
  sub2_scope   sub2_scp2(  top_scp1,  "sub2_scp2",   __FILE__, __LINE__);
  subsub_scope subsub_scp1(sub1_scp1, "subsub_scp1", __FILE__, __LINE__);

  std::string s1i = top_scp1.name();     // "top_scp1"
  std::string s2i = sub1_scp1.name();    // "sub1_scp1"
  std::string s3i = sub2_scp1.name();    // "sub2_scp1"
  std::string s4i = sub2_scp2.name();    // "sub2_scp2"
  std::string s4i = subsub_scp1.name(); // "subsub_scp1"

  std::string s1t = top_scp1.type_name();     // "top_scope"
  std::string s2t = sub1_scp1.type_name();    // "top_scope::sub1_scope"
  std::string s3t = sub2_scp1.type_name();    // "top_scope::sub2_scope"
  std::string s4t = sub2_scp2.type_name();    // "top_scope::sub2_scope"
  std::string s4t = subsub_scp1.type_name(); // "top_scope::sub1_scope::subsub_scope"
}
```

Listing 4.7: Instances of scope types of Listing 4.5

4.3 Coverage groups (covergroups)

The actual measurment of functional coverage is organized in covergroup classes. These covergroup classes are derived from class `fc4sc::covergroup`. Each of these covergroup classes can be instantiated in several objects. These objects are often member variables (attributes) of scoreboard classes.

The kind of declation of a covergroup class depends on the scope the covergroup belongs to.

4.3.1 Declaration for default scope

If the `default_scope` is used the declaration of the covergroup class needs a constructor that should be declared using the `CG_CONS` macro (Listing 4.8). The only argument of this macro is the name of the covergroup class.

```
class my_covergroup : public fc4sc::covergroup
{
public:
  CG_CONS(my_covergroup) {}
};
```

Listing 4.8: Constructor of a covergroup class in `default_scope`

The curly braces {} after the macro describe the function body of the constructor. If needed it contains code to initialize the members of the covergroup class.

The macro CG_CONS first declares the sample() function of fc4sc::covergroup to be the sample function of the covergroup base class. Then, it defines a constructor (Listing 4.9). If the macro CG_CONS is not used the sample() function must be declared in the same way as the macro does. In addition, the constructor of the covergroup class must call the constructor of the base class fc4sc::covergroup.

```
class my_covergroup : public fc4sc::covergroup
{
public:
  using covergroup::sample;
  my_covergroup(std::string inst_name = "",            // name of the covergroup instance
                const char* auto_inst_file_name = "", int auto_line = 1, // loc. of the inst.
                fc4sc::global* cntxt = fc4sc::global::getter()) :
    fc4sc::covergroup("my_covergroup",
                      __FILE__, __LINE__,              // location of the covergroup class
                      inst_name,                       // name of the covergroup instance
                      auto_inst_file_name, auto_line,  // location of the covergroup instance
                      cntxt)
  {}
};
```

Listing 4.9: Explicit definition of a constructor of a covergroup class in default_scope

Since all parameters of the constructor defined by CG_CONS have default values the constructor needs no explicit arguments. It is recommended to specify always the name of the instance as argument. The parameters for the location of the covergroup instance are typically set by the macros __FILE__ and __LINE__. If a specific context is used it must be specified as last argument. Because of the C++ mechanism for default parameters, the context can only be specified together with the instance location.

```
void f()
{
  my_covergroup cg1("cg1");
  my_covergroup cg2("cg2", __FILE__, __LINE__);

  fc4sc::global* cntxt = fc4sc::global::create_new_context();
  my_covergroup cg3("cg3", __FILE__, __LINE__, cntxt);
}
```

Listing 4.10: Instantiation of a covergroup

4.3.2 Declaration inside specific scope class

There are two ways to specify a covergroup for a specific scope. Either, the covergroup class is a part of the scope class (explained in this section) or the covergroup class is specified separately (Section 4.3.3).

Listing 4.11 shows an example where the covergroup is a part of the scope class. For the definition of the constructor of the covergroup the macro CG_SCOPED_CONS is used which gets the name of the scope class as an additional argument. The covergroup objects (cg1 in Listing 4.11) are attributes of the scope class. They must be initialized during construction of the scope object. The scope constructor is, therefore, extended using the macro CG_SCOPED_INST() for each covergroup object.

```
class my_scope : public fc4sc::scope
{
public:
  SCOPE_DECL(my_scope)

  my_scope(std::string name_,
           const char* auto_inst_file_name = "", int auto_line = 1, // loc. of the scope inst.
           fc4sc::global* cntxt = fc4sc::global::getter()) :
    fc4sc::scope(name_,
                 __FILE__, __LINE__,              // location of the scope class (my_scope)
                 auto_inst_file_name, auto_line,  // location of the scope instance
                 cntxt),
    CG_SCOPED_INST(cg1)                           // construction of cg1
  {} // further initialisations of my_scope

  class my_covergroup : public fc4sc::covergroup
  {
  public:
    CG_SCOPED_CONS(my_covergroup, my_scope)
    {} // further initialisations of my_covergroup
  };

  my_covergroup cg1; // instance of my_covergroup
};
```

Listing 4.11: Specification of a covergroup inside a scope

The macro call `CG_SCOPED_CONS(my_covergroup, my_scope)` creates a constructor for `my_covergroup` that expects the scope and the name of the covergroup instance as well as the location of the instance. This constructor calls a constructor of `fc4sc::covergroup`.

The macro call `CG_SCOPED_INST(cg1)` creates a call to this constructor of `my_covergroup`. It passes a pointer to this scope object and the name and location of the covergroup instance.

Listing 4.12 shows the declaration of a covergroup inside a scope without using the macros.

```
class my_scope : public fc4sc::scope
{
public:
  SCOPE_DECL(my_scope)

  my_scope(std::string name_,
           const char* auto_inst_file_name = "", int auto_line = 1, // loc. of the scope inst.
           fc4sc::global* cntxt = fc4sc::global::getter()) :
    fc4sc::scope(name_,
                 __FILE__, __LINE__,              // location of the scope class (my_scope)
                 auto_inst_file_name, auto_line,  // location of the scope instance
                 cntxt),
    cg1(*this,                                    // this scope
        "cg1",                                    // name of covergroup instance
        __FILE__, __LINE__)                       // loc. of covergroup instance
  {} // further initialisations of my_scope

  class my_covergroup : public fc4sc::covergroup
  {
  public:
    using covergroup::sample;
    my_covergroup(fc4sc::scp_base& scp,             // scope
                  std::string inst_name = "",       // name of covergroup instance
                  const char* auto_inst_file_name = "", int auto_line = 1) : // loc. cg. inst.
      fc4sc::covergroup(scp,
                        "my_covergroup",
                        my_scope::scp_type_name(),
                        __FILE__, __LINE__,             // loc. of covergroup class
                        inst_name,                      // name of covergroup instance
                        auto_inst_file_name, auto_line) // loc. of covergroup instance
    {} // further initialisations of my_covergroup
  };

  my_covergroup cg1; // instantiation of my_covergroup
};
```

Listing 4.12: Explicit specification of a covergroup inside a scope

A covergroup can be a part of any scope type – top-level scope or sub scope.

4.3.3 Declaration outside specific scope class

If the covergroup class is declared outside a scope class exactly the same declaration for the covergroup is used as for a declaration inside a scope class. Since the covergroup instances are not inside the scope class, the constructor of the scope class does not initialize them (Listing 4.13). The constructor call of the covergroup gets the scope instance as a parameter.

```
class my_scope : public fc4sc::scope
{
public:
  SCOPE_DECL(my_scope)

  my_scope(std::string name_,
           const char* auto_inst_file_name = "", int auto_line = 1, // loc. of the scope inst.
           fc4sc::global* cntxt = fc4sc::global::getter()) :
    fc4sc::scope(name_,
                 __FILE__, __LINE__,              // location of the scope class (my_scope)
                 auto_inst_file_name, auto_line,  // location of the scope instance
                 cntxt)
  {} // further initialisations of my_scope
};

class my_covergroup : public fc4sc::covergroup
{
public:
  CG_SCOPED_CONS(my_covergroup, my_scope)
  {} // further initialisations of my_covergroup
};

void f()
{
  fc4sc::global* cntxt = fc4sc::global::create_new_context();

  my_scope      scp1("scp1", __FILE__, __LINE__, cntxt);  // create scope

  my_covergroup cg1(scp1, "cg1", __FILE__, __LINE__);     // create covergroup
}
```

Listing 4.13: Specification and instantiation of a covergroup outside a scope class declaration

Since `my_covergroup` is declared as a part of scope type `my_scope` (by `CG_SCOPED_CONS(my_covergroup, my_scope)`) the constructor of `my_covergroup`-objects requires an instance of `my_scope`.

A covergroup can belong to any scope – top-level scope or sub scope.

4.3.4 Extending construction of covergroups

If the construction of a covergroup needs additional parameters the macro calls to `CG_CONS` or `CG_SCOPED_CONS` can be extended by those parameters. Because of the C++ mechanism for default parameters each additional parameter needs a default value.

```
class my_covergroup : public covergroup
{
public:
  CG_CONS(my_covergroup, int param = 0)
  {
    // handle param
  }
};

void f()
{
  fc4sc::global* cntxt = fc4sc::global::create_new_context();
  my_covergroup cg1("cg1", __FILE__, __LINE__, cntxt, 5); // param gets value 5
}
```

Listing 4.14: Additional parameter for a constructor of a covergroup class

In addition to the constructor declared by `CG_CONS` or `CG_SCOPED_CONS` further constructors can be declared

that hand over additional parameters to the covergroup object. It is necessary to call the constructor of the base class `covergroup` that gets the object's name and the context or the scope as a parameter. It is *not* allowed to use `CG_CONS` or `CG_SCOPED_CONS` more than one time in a class declaration. The parameters of the overloaded constructor must be chosen carefully to prevent ambiguities when calling the constructor. In Listing 4.15 the parameters of the additional constructor have no default value to prevent those ambiguities.

```
class my_covergroup : public covergroup
{
public:
  CG_CONS(my_covergroup) {}
  my_covergroup(std::string inst_name,
                const char* auto_inst_file_name, int auto_line,
                fc4sc::global* cntxt,
                int param) :          // additional parameter
    fc4sc::covergroup("my_covergroup",
                      __FILE__, __LINE__,
                      inst_name,
                      auto_inst_file_name, auto_line,
                      cntxt)
  {
    // handle param
  }
};
```

Listing 4.15: Additional constructor of a covergroup class (similar for covergroups in a scope)

To ensure the call of the constructor of the base class `fc4sc::covergroup` constructor delegation might be used as demonstrated in Listing 4.16.

```
class my_covergroup : public fc4sc::covergroup
{
public:
  CG_CONS(my_covergroup) {}
  my_covergroup(std::string inst_name,
                const char* auto_inst_file_name, int auto_line,
                fc4sc::global* cntxt,
                int param) :          // additional parameter
    my_covergroup(inst_name,
                  auto_inst_file_name, auto_line,
                  cntxt)
  {
    // handle param
  }
};
```

Listing 4.16: Additional constructor of a covergroup class using delegation (similar for covergroups in a scope)

Table 4.1 summarizes all constructors of class `fc4sc::covergroup`. The first one is used by `CG_CONS`, the last one is used by `CG_SCOPED_CONS`.

Table 4.1: Constructors of `class covergroup`

`covergroup(const char* type_name,`
`const char* file_name = "",`
`int line = 1,`
`const std::string& inst_name = "",`
`const char* inst_file_name = "",`
`int inst_line = 1,`
`fc4sc::global* cntxt = fc4sc::global::getter())`
	`type_name`: name of the class
	`file_name`, `line`: location of the class in code
	`inst_name`: name of the object
	`inst_file_name`, `inst_line`: location of the object in code
	`cntxt`: context

`covergroup(const char* type_name,`
`std::string scp_type_name,`
`const char* file_name = "",`
`int line = 1,`
`const std::string& inst_name = "",`
`const char* inst_file_name = "",`
`int inst_line = 1,`
`fc4sc::global* cntxt = fc4sc::global::getter())`
	additional specification of the name of the scope class `scp_type_name`

`covergroup(scp_base& mod,`
`const char* type_name,`
`std::string scp_type_name,`
`const char* file_name = "",`
`int line = 1,`
`const std::string& inst_name = "",`
`const char* inst_file_name = "",`
`int inst_line = 1)`
	specification of scope instance `mod` instead of context (Each scope instance is related to a context.)

4.3.5 Declaration of variables

A covergroup class must contain all the data for which the coverage should be measured as member variables (attributes). The class that instantiate a covergroup class is responsible to copy the appropriate data to these member variables.

The measurement of the coverage of a covergroup instance is implicitly started with the construction of the covergroup object. It can be stopped by the member function `stop()` and restarted with `start()`. More details are explained in Section 5.

To sample the data for the measurement of the coverage the function `sample()` must be called. This call evaluates all member variables. The data to be measured must, hence, be copied to the member variables first (Listing 4.17). Copying the data and sampling it can be implemented in an additional function (Listing 4.18).

```
class my_covergroup : public fc4sc::covergroup
{
public:
  CG_CONS(my_covergroup) {}
  uint8_t a;
  uint8_t b;
};

void f()
{
  // e.g. in scoreboard
  my_covergroup cg1; // vvvvvvvvvvvvvvvvvvvvvvvvvvvvvvvvv

  cg1.a = ...;       // copy values to cg1
  cg1.b = ...;
  cp1.sample();      // sample

  cg1.a = ...;       // copy values to cg1
  cg1.b = ...;
  cp1.sample();      // sample

  cg1.stop();        // ^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^

  cg1.start();       // vvvvvvvvvvvvvvvvvvvvvvvvvvvvvvvvv

  cg1.a = ...;       // copy values to cg1
  cg1.b = ...;
  cp1.sample();      // sample

  cg1.stop();        // ^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^
                     // stop at the end of simulation
}
```

Listing 4.17: Example covergroup class and object

```
class my_covergroup : public fc4sc::covergroup
{
public:
  CG_CONS(my_covergroup) {}
  uint8_t a;
  uint8_t b;
  void sample(uint8_t a, uint8_t b)
  {
    this->a = a; // copy values to covergroup object
    this->b = b;
    covergroup::sample();
  }
};

void f()
{
  // e.g. in scoreboard
  my_covergroup cg1;     // vvvvvvvvvvvvvvvvvvvvvvvvvvvvvvvvv

  cg1.sample( 8,  3);    // sample
  cg1.sample(19, 17);    // sample

  cg1.stop();            // ^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^

  cg1.start();           // vvvvvvvvvvvvvvvvvvvvvvvvvvvvvvvvv

  cp1.sample(-3, 14);    // sample

  cg1.stop();            // ^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^
                         // stop at the end of simulation
}
```

Listing 4.18: Example covergroup class with own sample function

4.4 Coverage points (coverpoints)

4.4.1 Defining coverpoints

Inside a covergroup class several coverage points (coverpoints) are declared. A coverpoint is characterized by its name, an expression that is to be evaluated, and the type of this expression. In addition, a condition can be specified under which the expression is evaluated.

It is recommended to use the macro `COVERPOINT` to declare a coverpoint:
`COVERPOINT(<type>, <name>, <expression> [, <condition>] )`

```
class my_covergroup : public covergroup
{
public:
  CG_CONS(my_covergroup) {}

  uint8_t abc;

  COVERPOINT(uint8_t, first_cp, abc, true)
  {
    bin<uint8_t>("a", 0)
  };
};
```

Listing 4.19: Example covergroup class with a coverpoint

The type of the expression must be an integral type - either a variant of `int` or `bool`.[1] Floating point numbers or aggregate types (arrays or structs) are not supported. Enums can be used with some limitations that are explained at the end of Section 4.4.2 (Listing 4.23, Listing 4.24).

The name of the covergroup must be a valid member name inside the covergroup class.

The condition must be of type `bool` at least after implicit type cast.

The condition and the expression are C++ expressions. They are evaluated in the `sample()` function. Therefore, condition and expression may also contain function calls. The expression is only evaluated if the condition evaluates to `true`. The condition can, hence, be used as a guard to the expression e.g. to prevent illegal array access.

Each coverpoint gets an initializer list that specifies its bins. A bin defines a value or a set of values. A bin is hit if the condition of the coverpoint is true and the expression of the coverpoint has a sample value that belongs to that bin. Each bin counts the number of hits.

Coverpoints without bins are not supported.

4.4.2 Defining bins

The bins of a coverpoint must be parameterized by the type of that coverpoint. Each bin has a name. The value(s) of the bin can be specified by parameters of the bin constructor:
`bin<type>(<name>, <value1>, <value2>, ...)`

```
class my_covergroup : public covergroup
{
public:
  CG_CONS(my_covergroup) {}

  uint8_t abc;

  COVERPOINT(uint8_t, first_cvp, abc, true) {
    bin<uint8_t>("zero", 0),
    bin<uint8_t>("one",  1),
    bin<uint8_t>("small", 2, 3, 4)
  };
};
```

Listing 4.20: Defining bins for a coverpoint

A continuous range of number can be specified by `interval(<from>, <to>)` with `<from>` $\leq$ `<to>`. This interval includes `<from>` as well as `<to>`. If an interval `interval(a, b)` is specified with `a` $>$ `b` then it is used as `interval(b, a)`.

The specification of intervals can be mixed with the specifications of single values. The ranges of all intervals and all single values must be of the type of the bin. It is allowed to specify overlapping intervals. They are merged such that internally only disjoint intervals are managed for a single bin.[2] When merged a message on `stderr` describes the merging.

[1] A type `T` is integral if `std::is_integral<T>::value` is `true`.

[2] The current version [Acc19] has a bug when intervals are merged. It is currently not recommended to use overlapping intervals. [Sch26] contains a patch for that bug.

```
class my_covergroup : public covergroup
{
public:
  CG_CONS(my_covergroup) {}

  uint8_t abc;

  COVERPOINT(uint8_t, first_cvp, abc, true) {
         bin<uint8_t>("zero", 0),
         bin<uint8_t>("one",  1),
         bin<uint8_t>("small", 2, 3, interval(5, 7), 9, interval(11, 20))
  };
};
```

Listing 4.21: Bins with intervals and single values

The sets of values defined for all bins of a coverpoint should be disjoint i.e. there should be no value that belongs to more than one bin. If a value belongs to two bins and this value is sampled then it counts for both bins. However, when using a cross with a coverpoint with overlapping bins the cross uses only one of the bins and not both. FC4SC chooses which bin is used for the cross such that the result might be unexpected. It is therefore recommended to use only disjoint bins.

In some cases, a long interval of numbers should be splitted in smaller intervals of the same size. For a convenient notation, FC4SC provides a class `bin_array` for this case. The constructor of `bin_array` gets as parameters the base name of the bins, the number of bins that should be generated, and an interval that should be splitted.

```
class my_covergroup : public covergroup
{
public:
  CG_CONS(my_covergroup) {}

  uint8_t abc;

  COVERPOINT( uint8_t, first_cvp, abc, true) {
    bin<       uint8_t>("zero", 0),
    bin<       uint8_t>("one",  1),
    bin_array<uint8_t>("small", 4, interval(2, 20))
  };
};
```

Listing 4.22: Coverpoint with `bin_array`

This constructor creates for the examples of Listing 4.22 a set of 4 bins `small[0]` ... `small[3]` with almost the same number of values from the specified interval. The bins `small[0]` ... `small[2]` contain the intervals $[2, 6]$, $[7, 11]$, and $[12, 16]$, resp. with 5 values each, and `small[3]` contains the interval $[17, 20]$ with the remaining 4 values. In general, the number of the first $(n-1)$ bins contain

$$number_of_bin_elements = \left\lfloor \frac{number_of_elements + 1}{number_of_bins} \right\rfloor \tag{1}$$

values and the last bin contains the remaining values.[3]

If the specified number of subintervals is larger than the number of elements of the interval to be splitted then only one bin with all elements is created.

It is possible to specify bins for illegal values using the class `illegal_bin`. This class is instantiated in the same way as regular bins. The values of illegal bins represent values that are not expected to occur during the simulation. Should an illegal bin been hit, an exception `fc4sc::illegal_bin_sample_exception`

[3] It is not clear if this kind of distribution is intended.

will be thrown, which can be caught outside `sample()`. `fc4sc::illegal_bin_sample_exception` is derived from `std::exception`. If the C++ macro `FC4SC_NO_THROW` is defined (e.g. by setting the compiler option `-DFC4SC_NO_THROW`) then the exception is caught inside FC4SC and an error message is printed.

If a value is contained in a regular bin and also in an illegal bin then this value is treated as illegal.

In some cases intervals belong to a bin but with some gaps. Using the method described above, such a bin can be described by intervals that range from gap to gap. For convenience, it is allowed to define values that are ignored in a coverpoint. Each coverpoint may contain an arbitrary number of `ignore_bin`. The values of each `ignore_bin` can be described in the same way as regular bins (i.e. using several single values and intervals).

If a value "hits" an ignore bin then this value will neither hit a regular nor an invalid bin. All values that are not contained in a bin of a coverpoint are also ignored.

During `sample()` the following steps are executed for each coverpoint:

1. evaluate condition, if it evaluate to false, skip the remaining steps
2. evaluate expression to get a value
3. if the value is contained in one of the ignore bins, skip the remaining steps
4. if the value is contained in one of the illegal bins, throw an exception or print an error message and skip the remaining steps
5. if the value is not contained in one of the regular bins, skip the remaining step
6. increment the hit counter for each bin that includes the value

When defining a bin, the values of a bin are handed over to the bin object by its constructor. In most cases, these values are, therefore, number literals e.g. `bin<int>("zero", 0)`. If member variables of the covergroup class are used (e.g. `bin<int>("abc", abc)`) they must be initialized before the coverpoint that contains the bin. It is not recommended to use member variables for this purpose. To create generic or parameterized bins, the strategies described in Section 4.8.1 should be used.

A coverpoint can have an arbitrary integral type as described in Section 4.4.1. Traditional enum types can be used because the coverpoint uses in this case the underlying integer data type (Listing 4.23). To use enum classes for a coverpoint these enum values must explicitely be casted to the underlying integer type (Listing 4.24). `interval()` descriptions for enum typed bins refer to the underlying integer type.

```
typedef enum {aaa, bbb, ccc} enum_t;

class my_covergroup : public fc4sc::covergroup
{
public:
  CG_CONS(my_covergroup) {}

  enum_t enum_var;

  COVERPOINT(int, enum_cp, enum_var, true)
  {
    bin<int>("aaa", aaa),
    bin<int>("bbb", bbb),
    bin<int>("ccc", ccc)
  };
};
```

Listing 4.23: Defining a coverpoint for an enum

```
enum class ec_impl {ddd, eee, fff};

class my_covergroup : public fc4sc::covergroup
{
public:
  CG_CONS(my_covergroup) {}

  ec_impl  ec_impl_var  {ec_impl::ddd};

  typedef std::underlying_type<ec_impl>::type ec_impl_t;

  COVERPOINT(ec_impl_t, enum_ec_impl_cp, static_cast<ec_impl_t>(ec_impl_var))
  {
    bin<ec_impl_t>("ddd", static_cast<ec_impl_t>(ec_impl::ddd)),
    bin<ec_impl_t>("eee", static_cast<ec_impl_t>(ec_impl::eee)),
    bin<ec_impl_t>("fff", static_cast<ec_impl_t>(ec_impl::fff))
  };
};
```

Listing 4.24: Defining a coverpoint for an enum class

4.4.3 Dynamic bin construction

Besides the static definition of bins as described above (Section 4.4.2) it is possible to specify functions that create bins during construction of coverpoints.

The constructor of `bin` allows single values and intervals of values as described above. It is possible to specify functions that return single values or intervals and to call those functions to define a bin (Listing 4.25).

```
class my_covergroup : public fc4sc::covergroup
{
public:
  CG_CONS(my_covergroup) {}

  uint8_t abc;

  uint8_t succ(uint8_t n)
  {
    return static_cast<uint8_t>(n+1);
  }

  fc4sc::interval_t<uint8_t> pred_succ(uint8_t n)
  {
    return interval(static_cast<uint8_t>(n-1), static_cast<uint8_t>(n+1));
  }

  COVERPOINT( uint8_t, bin_cp,  abc, true) {
    bin<uint8_t>("succ", succ(2), succ(7)),                    // 3, 8
    bin<uint8_t>("pair", pred_succ(10), pred_succ(20)),        // [9, 11], [19, 21]
    bin<uint8_t>("mix",  pred_succ(100), succ(20), 123, interval(200, 204))
                                                               // [99, 101], 21, 123, [200, 204]
  };
};
```

Listing 4.25: Coverpoint with a `bin` initialized dynamically

It is also allowed to construct a bin with a vector of intervals. Such a vector can be created inside a function (Listing 4.26). This way, a bin can be created by a function with the same expressive power as with an explicit/static specification. Single values are expressed as intervals with equal lower and upper bound.

```
class my_covergroup : public fc4sc::covergroup
{
public:
  CG_CONS(my_covergroup) {}

  uint8_t abc;

  std::vector< fc4sc::interval_t<uint8_t> > log_ranges(uint8_t start, int8_t num)
  {
    std::vector< fc4sc::interval_t<uint8_t> > result;
    uint8_t a = start == 0 ? 0 : 1<<start;
    uint8_t b = start == 0 ? 1 : (a*2)-1;
    for (uint8_t i = 0; i < num; i++)
    {
      fc4sc::interval_t<uint8_t> intrvl = interval(a, b);
      result.push_back(intrvl);
      a = b+1;
      b = (2*b) + 1;
    }
    return result;
  }

  COVERPOINT( uint8_t, bin1_cp,  abc, true) {
    bin<uint8_t>("many", log_ranges(0, 4))  // [0, 1], [2, 3], [4, 7], [8, 15]
  };

  COVERPOINT( uint8_t, bin2_cp,  abc, true) {
    bin<uint8_t>("many", log_ranges(0, 4), log_ranges(5, 2), 255)
                                  // [0, 1], [2, 3], [4, 7], [8, 15], [32, 63], [64, 127], 255
  };
};
```

Listing 4.26: Coverpoints with a `bin` initialized dynamically by a vector of intervals

A `bin_array` can be constructed in a similar way. There is a second constructor for `bin_array<T>` that expects after its name a `std::vector<T>`. Each element of this vector becomes a separate bin. This vector can be generated by a function such that larger number of bins can be created.

```
class my_covergroup : public fc4sc::covergroup
{
public:
  CG_CONS(my_covergroup) {}

  uint8_t abc;

  std::vector<uint8_t> even_numbers(uint8_t max)
  {
    std::vector<uint8_t> result;
    for (uint8_t i = 2; i <= max; i+=2)
    {
      result.push_back(i);
    }
    return result;
  }

  COVERPOINT( uint8_t, first_cp, abc, true) {
    bin<       uint8_t>("zero", 0),
    bin<       uint8_t>("one",  1),
    bin_array<uint8_t>("even", even_numbers(10))
    // bin even[0]:  2
    // bin even[1]:  4
    // bin even[2]:  6
    // bin even[3]:  8
    // bin even[4]: 10
  };
};
```

Listing 4.27: Coverpoint with `bin_array` initialized by a `vector`

If the bins should contain intervals instead of single values another constructor of `bin_array<T>` is used. It expects a `std:vector` with elements of type `fc4sc::interval_t<T>` as parameter. This constructor creates a bin for each interval. To include bins with single values in such a coverpoint intervals with `from=to` can be created that contain only one element.

```
class my_covergroup : public fc4sc::covergroup
{
public:
  CG_CONS(my_covergroup) {}

  uint8_t abc;

  std::vector< fc4sc::interval_t<uint8_t> > log_ranges(uint8_t start, int8_t num)
  {
    std::vector< fc4sc::interval_t<uint8_t> > result;
    uint8_t a = start == 0 ? 0 : 1<<start;
    uint8_t b = start == 0 ? 1 : (a*2)-1;
    for (uint8_t i = 0; i < num; i++)
    {
      fc4sc::interval_t<uint8_t> intrvl = interval(a, b);
      result.push_back(intrvl);
      a = b+1;
      b = (2*b) + 1;
    }
    return result;
  }

  COVERPOINT( uint8_t, ba2_cp, abc, true) {
    bin_array<uint8_t>("log", log_ranges(0, 4))
  };
  // bin log[0]: [0,  1];
  // bin log[1]: [2,  3];
  // bin log[2]: [4,  7];
  // bin log[3]: [8, 15];
};
```

Listing 4.28: Coverpoint with `bin_array` initialized by a `vector` of intervals

After the creation of a covergroup object a coverpoint of the covergroup can be extended by further bins.[4] After creating a suitable bin object this object is handed over to the coverpoint.

```
void f()
{
  my_covergroup cg1;

  bin<uint8_t> new_bin = bin<uint8_t>("max", 255);
  new_bin.add_to_cvp(cg1.first_cvp);

  bin<uint8_t>("almost max", 254).add_to_cvp(cg1.first_cvp);
}
```

Listing 4.29: Adding a bin to a coverpoint (based on the covergroup of Listing 4.21)

4.5 Cross coverage

FC4SC allows the definition of a cross product of two or more coverpoints. It is defined by these coverpoints and may have a name. The cross product contains bins for all combinations of all regular bins of the involved coverpoints. In the definition of a cross the types must be the types of the involved coverpoints. The number of bins of a cross product is the product of the number of regular bins of its coverpoints.

[4] Since it is possible to add bins during runtime one could consider to extend FC4SC to support coverpoint definitions without bins in the future. This would allow a complete bin definition for a coverpoint during runtime.

```
class my_covergroup : public fc4sc::covergroup
{
public:
  CG_CONS(my_covergroup) {}

  uint8_t abc;
  int8_t  def;
  bool    c1;
  bool    c2;

  COVERPOINT(uint8_t, abc_cp, abc, c1) {
         bin<uint8_t>("zero", 0),
         bin<uint8_t>("one",  1),
         bin<uint8_t>("small", 2)
  };
  COVERPOINT(int8_t, def_cp, def, c2) {
         bin<int8_t>("neutral", 0),
         bin<int8_t>("pos",    +1),
         bin<int8_t>("neg",    -1)
  };
  cross<uint8_t, int8_t> abc_def_cr =
  cross<uint8_t, int8_t>(this, "abc x def", &abc_cp, &def_cp);
};
```

Listing 4.30: Cross defined by two coverpoints

A bin of a cross is hit only if the bins of all the coverpoints are hit i.e. when the condition of all coverpoints is true at the same sampling point. In the example of Listing 4.30 `c1` and `c2` must be `true` to hit a cross bin. If `c1=c2=true`, `abc=0`, and `def=0` then the cross bin `("zero", "neutral")` (or $(0, 0)$) is hit.

4.6 Dynamic creation of covergroups

Section 4.3 describes that covergroup types are represented as C++ classes. They must, therefore, be defined at compile time. FC4SC allows the definition of covergroup types at runtime. An instance of the class `dynamic_covergroup_factory` represents a covergroup type. This covergroup type must then be populated with coverpoints and crosses. The `dynamic_covergroup_factory` provides the member function `create_coverpoint` to create a coverpoint and add it to the covergroup. The name and type of the coverpoint must be defined together with callables (e.g. a function, a lamda function, or a functor) that define the sampling value and, optionally, the condition. The return value of the condition function must be `bool`. Bins with the suitable value type are added to the coverpoint by `create_bin()`, `create_illegal_bin()`, and `create_ignore_bin()`. There are different versions of `create_bin_array()` to create an array of regular bins in the same way as described in Section 4.4.2 and Section 4.4.3. An arbitrary number of coverpoints can be combined to a cross that is added to the covergroup type by `create_cross()`.

```
void f()
{
  fc4sc::dynamic_covergroup_factory cvg("cvg");

  fc4sc::dynamic_coverpoint_factory<int(int, int), bool(int)>
    cvp_sum = cvg.create_coverpoint< int(int, int), // sampling function
                                     bool(int) >    // condition function
                                   ( "sum",                          // name
                                     [](int x, int y) {return x+y;}, // sampling function
                                     [](int x) {return x == -1;} );  // condition function
  cvp_sum.create_bin("ZERO", 0);
  cvp_sum.create_bin("ONE",  1);

  fc4sc::dynamic_coverpoint_factory<int(int, int), bool()>
    cvp_diff = cvg.create_coverpoint< int(int, int) >
                                    ("diff", [](int x, int y) {return x-y;} );

  cvp_diff.create_bin("ZERO", 0);
  cvp_diff.create_bin("TWO",  2);

  cvg.add_cross("cr", cvp_sum, cvp_diff);
}
```

Listing 4.31: Dynamic creation of a covergroup type

After creation of a covergroup type dynamically, a covergroup instance can be created from this type as an instance of class `dynamic_covergroup`. The constructor gets the object that represents the covergroup type as a parameter. In contrast to the statically defined covergroup types, the dynamically definded covergroups do not contain the variables to be sampled. They must be created separately and binded to the covergroup. Since the binding functions are members of `dynamic_coverpoint_factory`, the objects that represent the coverpoints must be available for binding the variables of each covergroup instance.

```
void f()
{
  fc4sc::global* cntxt = fc4sc::global::create_new_context();

  fc4sc::dynamic_covergroup_factory cvg("cvg"); // create a covergroup type
  // define coverpoints and crosses of that covergroup type

  // create covergroup instance cvg1 of covergroup type cvg
  fc4sc::dynamic_covergroup cvg1(cvg, "cvg1", __FILE__, __LINE__, cntxt);

  int v1, v2; // variables to be sampled

  cvp_sum.bind_sample(   cvg1, v1, v2);
  cvp_sum.bind_condition(cvg1, v2);

  cvp_diff.bind_sample(cvg1, v1, v2);

  // binding complete, usage can start now
  v1 =  1;
  v2 = -1;

  cvg1.sample();
}
```

Listing 4.32: Creation of an instance from a dynamically created covergroup type and sampling values

A dynamically created covergroup cannot belong to a statically created scope but to a dynamically created scope.

4.7 Dynamic creation of scopes

Scopes can be created dynamically i.e. the covergroups and sub scopes that belong to a scope can be defined during runtime. At first, the used scope type must be defined as instance of class `dynamic_scope_factory`. It is mainly defined by its name. Then, the types of the covergroups are defined as `dynamic_covergroup_factory` that is constructed with the type of the scope that covergroup type belongs to.

```
void f()
{
  fc4sc::dynamic_scope_factory       scp("scp");
  fc4sc::dynamic_covergroup_factory cvg(scp, "cvg");
  // create coverpoints for this covergroup cvg
}
```

Listing 4.33: Dynamic creation of a scope type and a covergroup type

To create the instances of the scope and its covergroups, it must first be defined by `add_covergroup()` which instances of covergroups belong to this scope. When the scope type is instantiated as `dynamic_scope` then the covergroup instances of this scope are created. Please note that `dynamic_scope` is a class derived from `scope`. To get access to a covergroup instance of a scope instance the function `get_covergroup()` can be used.

```
void f()
{
  fc4sc::dynamic_scope_factory       scp("scp");
  fc4sc::dynamic_covergroup_factory cvg(scp, "cvg");

  scp.add_covergroup(cvg, "cvg1"); // instance cvg1 of cvg belongs to scp
  fc4sc::global* cntxt = fc4sc::global::create_new_context();
  fc4sc::dynamic_scope scp1(scp, "scp1", __FILE__, __LINE__, cntxt); // creates scp1 and cvg1

  fc4sc::dynamic_covergroup& cvg1 = dynamic_cast<fc4sc::dynamic_covergroup&>(scp1.
      get_covergroup("cvg1"));
}
```

Listing 4.34: Dynamic creation of a scope instance and a covergroup instances

To define scope types as sub scopes of another scope types the parent scope type must be specified when constructing the sub scope.

```
void f()
{
  fc4sc::dynamic_scope_factory top_scp("top_scp");
  fc4sc::dynamic_scope_factory sub1_scp(top_scp, "sub1_scp");
  fc4sc::dynamic_scope_factory sub2_scp(top_scp, "sub2_scp");
  fc4sc::dynamic_scope_factory subsub_scp(sub1_scp, "subsub_scp");
}
```

Listing 4.35: Dynamic creation of a scope type hierarchy

To define the scope instances of sub scopes the function `add_child()` must be used as demonstrated in Listing 4.36.

```
void f()
{
  // define scope type hierarchy
  fc4sc::dynamic_scope_factory top_scp("top_scp");
  fc4sc::dynamic_scope_factory sub1_scp(top_scp, "sub1_scp");
  fc4sc::dynamic_scope_factory sub2_scp(top_scp, "sub2_scp");
  fc4sc::dynamic_scope_factory subsub_scp(sub1_scp, "subsub_scp");

  // define scope instances
  top_scp.add_child(sub1_scp,    "sub1_scp1");
  top_scp.add_child(sub2_scp,    "sub2_scp1");
  top_scp.add_child(sub2_scp,    "sub2_scp2");
  sub1_scp.add_child(subsub_scp, "subsub_scp1");

  // define covergroup instances
  fc4sc::dynamic_covergroup_factory cvg1(top_scp,  "cvg1");
  fc4sc::dynamic_covergroup_factory cvg2(sub1_scp, "cvg2");
  fc4sc::dynamic_covergroup_factory cvg3(sub2_scp, "cvg3");
  // define further covergroups
  // create coverpoints for these covergroups

  // create top_scp1 and all child scopes (recursively) with their covergroups
  fc4sc::dynamic_scope top_scp1(top_scp, "top_scp1", __FILE__, __LINE__, cntxt);

  cvg_base cvg1 = top_scp1.get_covergroup("cvg1");
  scp_base scp1 = top_scp1.get_child("sub1_scp1");
  cvg_base cvg2 = scp1.get_covergroup("cvg2");
  cvg_base cvg3 = top_scp1.get_child("sub2_scp1").get_covergroup("cvg3");
}
```

Listing 4.36: Dynamic creation of a scope instances and covergroup instances

4.8 Common issues and solutions

4.8.1 Coverpoints for arbitrary relations and types

In some applications a relation of two or more values should be covered. Since FC4SC is only capable to measure the coverage of numbers a function can be declared that transforms the relation into a number.

If e.g. the relation “greater”, “less”, and “equal” of two numbers should be covered a function `int8_t compare(a, b)` can be defined and evaluated.

```
class my_covergroup : public fc4sc::covergroup
{
public:
  CG_CONS(my_covergroup) {}

  uint8_t a;
  uint8_t b;

  int8_t compare(uint8_t a, uint8_t b)
  {
    if      (a<b) return -1;
    else if (a>b) return +1;
    else          return  0;
  }

  COVERPOINT(int8_t, a_b_rel_cvp, compare(a, b), true) {
         bin<int8_t>("a<b",  -1),
         bin<int8_t>("a>b",  +1),
         bin<int8_t>("a==b",  0)
  };
};
```

Listing 4.37: Describing a relation in coverpoints and bins using an auxiliar function

Coverage points for floating point numbers or other non-integral data types can also be defined using this approach.

4.8.2 Conditional cross coverage

The definition of cross coverages does not contain a condition. There is an implicit condition by the conjuction of the conditions of the used coverpoints: a cross bin is hit only if all related bins of the coverpoints are hit. If the cross should be used under a more restrictive condition an auxiliary coverpoint is defined. It contains only one bin that represents the required condition. This auxiliary coverpoint is then added to the cross coverage. The bins of the cross coverage are only hit if the bin of the auxiliary coverpoint is hit. The number of bins of the cross coverage is not changed by adding such an auxiliary coverpoint.

```
class my_covergroup : public fc4sc::covergroup
{
public:
  CG_CONS(my_covergroup) {}

  uint8_t abc;
  int8_t  def;
  int8_t  ghi;
  bool    c1;
  bool    c2;

  COVERPOINT(uint8_t, abc_cp, abc, c1) {
         bin<uint8_t>("zero", 0),
         bin<uint8_t>("one",  1),
         bin<uint8_t>("small", 2)
  };
  COVERPOINT(int8_t, def_cp, def, c2) {
         bin<int8_t>("neutral", 0),
         bin<int8_t>("pos",  1),
         bin<int8_t>("neg", -1)
  };
  COVERPOINT(bool, cond_cp, ghi < def, ghi > 0) {
         bin<bool>("dummy", true)
  };
  cross<uint8_t, int8_t> abc_def_cr =
  cross<uint8_t, int8_t>(this, "abc x def", &abc_cp, &def_cp);

  cross<uint8_t, int8_t, bool> abc_def_cond_cr =
  cross<uint8_t, int8_t, bool>(this, "abc x def (cond)", &abc_cp, &def_cp, &cond_cp);
};
```

Listing 4.38: Cross with condition defined by coverpoint `cond_cp`

The cross `abc_def_cr` of Listing 4.38 contains $3 \cdot 3 = 9$ bins and the cross `abc_def_cond_cr` contains also $3 \cdot 3 \cdot 1 = 9$ bins. A cross bins can only be hit if all conditions of the coverpoints are hit, i.e. $c1 \wedge c2$ for `abc_def_cr` and $c1 \wedge c2 \wedge (ghi > 0)$ for `abc_def_cond_cr`. Moreover, each coverpoint must hit one of its regular bins, i.e. $(abc \in \{0,1,2\}) \wedge (def \in \{0,1,-1\})$ for `abc_def_cr` and $(abc \in \{0,1,2\}) \wedge (def \in \{0,1,-1\}) \wedge (ghi < def)$ for `abc_def_cond_cr`. Thus, when adding the coverpoint `cond_cp` to `abc_def_cr` (which leads to `abc_def_cond_cr`) the condition $(ghi > 0) \wedge (ghi < def)$ is added.

5 Controlling functional coverage

Each covergroup has a member function `sample()`. If this function is called for a covergroup object then every coverpoint and cross of this object will be sampled if they are not disabled. FC4SC assumes that the structure of scopes, covergroups, coverpoints, and crosses will not change after the first call of `sample()`.

It is possible to sample just a single coverpoint or cross of a covergroup by calling `sample()` for this coverpoint or cross, respectively. If `sample()` is called for a coverpoint the crosses that use this coverpoint are not sampled automatically. If `sample()` is called for a cross then the coverpoints related to this cross are not sampled automatically. Sampling a cross is based on the sampling result of the previous sampling of the related coverpoints. To hit a cross bin, a regular bin of each related coverpoint must have been hit during the recent run of `sample()` of this coverpoint. If more than one bin of a related coverpoint has been hit before then only the last one is used for selecting the cross bin. Calling `sample()` for a disabled coverpoint (i.e. collect flag is false) does not count as a prevoius/recent sampling.

To avoid inconsistent results it has to be ensured that all related coverpoints are sampled before a cross is sampled. In complex sampling scenarios it has to be considered to split a covergroup class in smaller ones and to sample only complete covergroups.

Calling the member function `stop()` for a coverpoint or cross sets the collect flag of the coverpoint or cross

to false (i.e. disable) which prevents sampling in subsequent calls to `sample()`. Calling the member function `start()` for a coverpoint or cross sets the collect flag of the coverpoint or cross to true (i.e. enable) which allows sampling in subsequent calls to `sample()`. The functions `start()` and `stop()` of a covergroup object enable or disable all its coverpoints and crosses by setting their collect flags to true or false, respectively. Calling `sample()` for a covergroup instance, coverpoint, or cross has no effect for disabled coverpoints and crosses (collect flag is false).

To enable or disable a complete covergroup without changing the enable/disable status (i.e. the collect flag) of all coverpoints and crosses the enable status of the covergroup can be changed. Listing 5.1 gives an example. The enable status of the covergroup does not only influence the `sample()` function but also the generation of the coverage status (`get_inst_coverage()`). Sampling a covergroup when it is not enabled leads to an error message. Setting the enable status of a covergroup is aimed to prevent unintended sampling rather than skip some sampling points.

```
class my_covergroup : public fc4sc::covergroup
{
//...
};

void f()
{
  my_covergroup cg1;
  fc4sc::cvg_base_data_model* cvg_data = cg1.get_cvg_data();
  cvg_data->enable = true;  // enable covergroup
  cvg_data->enable = false; // disable covergroup
  if (cg1.is_enabled())     // check if cg1 is enabled
  { }
}
```

Listing 5.1: Enabling and disabling a covergroup

Table 5.1: Function of `class covergroup` for controlling coverage

`void sample()`	sample the coverpoints and crosses of the covergroup
`void start()`	Enable sampling of all coverpoints and crosses of the covergroup by setting the collect flags to true. The enable status of the covergroup remains unchanged.
`void stop()`	Disable sampling of all coverpoints and crosses of the covergroup by setting the collect flags to false. The enable status of the covergroup remains unchanged.
`void disable_cvg()`	not implemented, throws always an exception (`std::string`)
`bool is_enabled()`	return enable status of the covergroup

Table 5.2: Functions of `class coverpoint` for controlling coverage

`void sample()`	sample the coverpoint
`void start()`	Enable sampling of the coverpoint by setting the collect flag to true.
`void stop()`	Disable sampling of the coverpoint by setting the collect flag to false.

Table 5.3: Functions of `class cross` for controlling coverage

`void sample()`	sample the cross
`void start()`	Enables sampling of the cross by setting the collect flag to true.
`void stop()`	Disables sampling of the cross by setting the collect flag to false.

If the project is compiled with the defined macro `FC4SC_DISABLE_SAMPLING` e.g. by a compiler option `-DFC4SC_DISABLE_SAMPLING` then all sampling is disabled.

6 Data structure of a context

6.1 Overview

The complete data for measuring functional coverage is maintained in a context (Figure 6.1). The context class has a vector `scps_data` with references to objects of `scp_metadata` that maintain all scope classes and a vector `top_scps` with references to objects of `scp_base_data_model` that maintains all scope instances. The `scp_base_data_model` has a reference to `scp_metadata` of its scope class. The `scp_metadata` has references to `scp_base_data_model` of all objects of its scope class.

In addition, the `scp_base_data_model` has references to all of its covergroup instances that are maintained in `cvg_base_data_model`. The `cvg_base_data_model` has references to all coverpoints and crosses that are maintained by `cvp_base_data_model` as well as to `cvg_metadata` that manages data specific to covergroup classes.

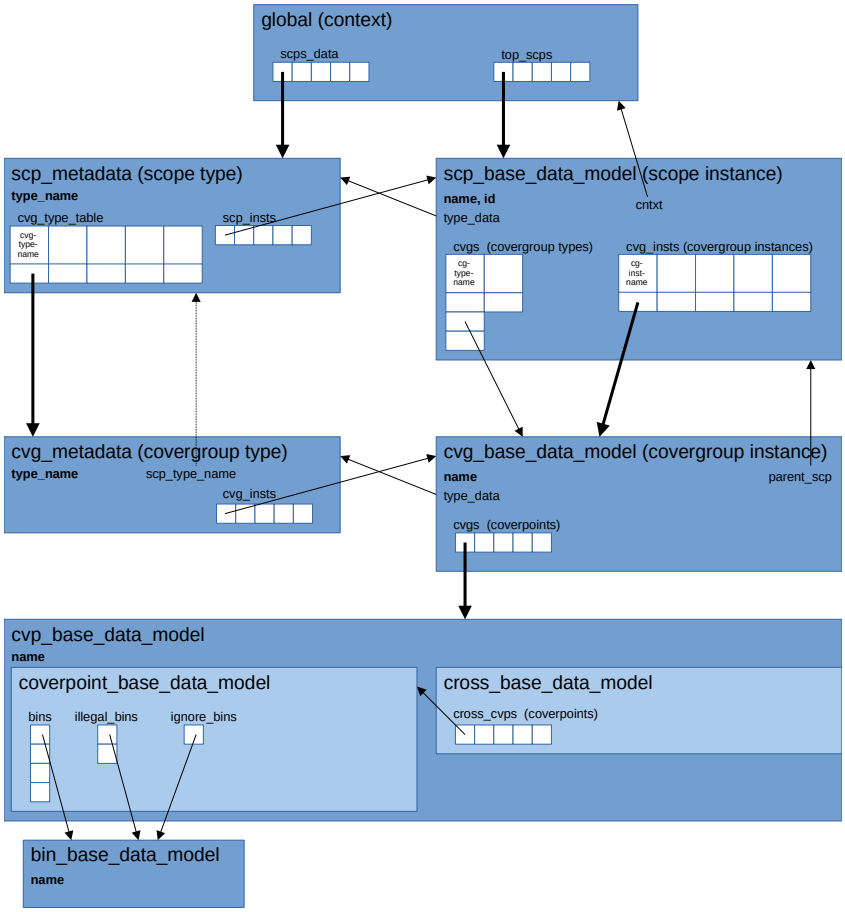


Figure 6.1: Classes to maintain coverage information for each context

6.2 Context (global)

The most important public members of a context (class `fc4sc::global`) are shown in Table 6.1 (non-static) and Table 6.2 (static).

Table 6.1: Important public members of class `global` (`fc4sc_master.hpp`)

`std::unordered_map<std::string,scp_metadata*>`	`scps_data`	map of all scope types
`std::vector<scp_base_data_model*>`	`top_scps`	vector of all scope instances
`default_scope*`	`dflt_scp`	pointer to default scope
`std::unordered_map<std::string, unsigned int>`	`file_name_to_id`	maps file names to IDs
`std::unordered_map<unsigned int, std::string>`	`file_id_to_name`	maps file IDs to names

Table 6.2: Important public static members of class `global` (`fc4sc_master.hpp`). The parameter `cntxt` has always the default value of the global context as returned by `fc4sc::global::getter()`.

`global* getter()`	global context
`global* create_new_context()`	creates a new context
`void delete_context(global* cntxt)`	delete a context
`scp_base* get_default_scope(fc4sc::global* cntxt)`	default scope instance

`cvg_type_option& type_option(`
`const std::string& scp_type,`
`const std::string& type, fc4sc::global* cntxt)`
	type option of the covergroup type "`type`" inside scope type "`scp_type`"

`std::unordered_map<std::string,scp_metadata*>&` `get_scopes_data(fc4sc::global* cntxt)`	`scps_data`
`std::vector<scp_base_data_model*>&` `get_top_scopes(fc4sc::global* cntxt)`	`top_scps`
`bool is_empty(fc4sc::global* cntxt)`	`scps_data.empty()`
`const std::unordered_map<unsigned int, std::string>&` `get_file_id_to_name_table(fc4sc::global* cntxt)`	returns `file_id_to_name`

File names are referenced in the data structure by file IDs (`unsigned int`). Each context manages a map that relates file IDs and file names. Listing 6.1 shows an example to get the name for an ID.

```
std::string get_file_name(unsigned int file_id, fc4sc::global* cntxt)
{
  std::unordered_map<unsigned int, std::string>::const_iterator file_it =
                                  fc4sc::global::get_file_id_to_name_table(cntxt).find(file_id);
  if (file_it != fc4sc::global::get_file_id_to_name_table(cntxt).cend())
  {
    return file_it->second;
  }
  return "";
}
```

Listing 6.1: Getting the file name from a file ID

6.3 Scope types

Table 6.3: Important public members of class `scp_metadata` (`fc4sc_base.hpp`)

`std::string type_name`	name of the scope type
`unsigned int file_id`	ID of the file where this scope type is declared
`uint32_t line`	line no. in the file where this scope type is declared
`std::vector<scp_base_data_model*> scp_insts`	all instances of this scope type (Table 6.4)
`std::unordered_map<std::string, cvg_metadata*>` `cvg_type_table`	all covergroup types that belong to this scope type (Table 6.5)

6.4 Scope instances

Table 6.4: Important public members of class `scp_base_data_model` (`fc4sc_base.hpp`)

`std::string`	`name`	name of the scope instance
`unsigned int`	`inst_file_id`	ID of the file where this instance is instantiated
`uint32_t`	`inst_line`	line no. in the file where this instance is instantiated
`unsigned int`	`instance_id`	ID of this scope instance, unique within context
`scp_metadata*`	`type_data`	attributes of the scope type of this instance (Table 6.3)
`global*`	`cntxt`	context where this scope instance belongs to
`scp_base_data_model*`	`parent_scp`	reference to the parent scope (`nullptr` if not existing)
`std::unordered_map<std::string, scp_base_data_model*>`	`child_scp_insts`	all sub (child) scope instances that belongs to this scope instance
`std::unordered_map<std::string, std::vector<scp_base_data_model*>>`	`child_scps`	map of sub (child) scope type names to all scope instances that belongs to this scope type
`std::unordered_map<std::string, cvg_base_data_model*>`	`cvg_insts`	all covergroup instances that belongs to this scope instance (Table 6.7)
`std::unordered_map<std::string, std::vector<cvg_base_data_model*>>`	`cvgs`	map of covergroup type names to all covergroup instances that belongs to this covergroup type and this scope instance (Table 6.7)

6.5 Covergroup types

Table 6.5: Important public members of class `cvg_metadata` (`fc4sc_base.hpp`)

`std::string`	`type_name`	name of this covergroup type
`cvg_type_option`	`type_option`	see Table 6.6
`std::string`	`scp_type_name`	name of the scope type where this covergroup type belongs to
`unsigned int`	`file_id`	ID of the file where this type is declared
`uint32_t`	`line`	line no. in file where this type is declared
`std::vector<cvg_base_data_model*>`	`cvg_insts`	data of all covergroup instances of this covergroup type (Table 6.7)

Table 6.6: Attributes of class `cvg_type_option` (`fc4sc_options.hpp`)

`uint/uint64_t weight = 1`	weight of the covergroup type inside its scope
`uint/uint32_t goal = 100`	coverage goal of the covergroup type (in percent)
`bool merge_instances = false`	unused
`std::string comment = ""`	for UCIS XML file, only

Since the type `uint` is not available on all systems the patch of [Sch26] uses suitable types of `cstdint`. To mark those changes both types are shown in the tables.

6.6 Covergroup instances

Table 6.7: Attributes of class `cvg_base_data_model` (`fc4sc_base.hpp`)

`cvg_metadata*`	`type_data`	attributes of the covergroup class (Table 6.5)
`cvg_option`	`option`	see Table 6.8
`unsigned int`	`inst_file_id`	ID of the file where this object is instantiated
`uint32_t`	`inst_line`	line no. in file where this object is instantiated
`std::vector<cvp_base_data_model*>`	`cvps`	vector of all coverpoints and crosses
`bool`	`enable`	enable flag (Section 5)
`scp_base_data_model*`	`parent_scp`	pointer to the scope instance where the covergroup belongs to (Table 6.4)

Table 6.8: Attributes of class `cvg_option` (`fc4sc_options.hpp`)

`uint/uint64_t weight`	`= 1`	weight of the covergroup instance inside its covergroup type
`uint/uint32_t goal`	`= 100`	coverage goal of the covergroup instance (in percent)
`uint/uint64_t at_least`	`= 1`	for UCIS XML file, only
`uint/uint32_t auto_bin_max`	`= 10`	for UCIS XML file, only
`bool detect_overlap`	`= false`	for UCIS XML file, only
`uint/uint32_t cross_num_print_missing`	`= 0`	for UCIS XML file, only
`bool per_instance`	`= false`	for UCIS XML file, only
`bool get_inst_coverage`	`= false`	unused
`std::string comment`	`= ""`	for UCIS XML file, only

6.7 Coverpoint and cross instances

The class `cvp_base_data_model` does not contain much information. It is the base class of `coverpoint_base_data_model` for coverpoints and `cross_base_data_model` for crosses. Since both classes are abstract, the objects belong to the class `coverpoint_data_model` and `cross_data_model`, respectively, which are derived from those classes. A `dynamic_cast<>` can be used to determine the actual identity of a `cvp_base_data_model` pointer as shown in Listing 6.2.

```
void f()
{
  fc4sc::cvp_base_data_model* cp;
  cp = ...; // e.g. from cvg_base_data_model -> cvps[i]

  fc4sc::coverpoint_data_model* cpt = dynamic_cast<fc4sc::coverpoint_data_model*>(cp);
  fc4sc::cross_data_model*      cpc = dynamic_cast<fc4sc::cross_data_model*     >(cp);
  if (cpt != nullptr)
  {
    // cp is a coverpoint -> cpt
  }
  else if (cpc != nullptr)
  {
    // cp is a cross -> cpc
  }
}
```

Listing 6.2: Casting a pointer of `cvp_base_data_model`

Table 6.9: Members of class `coverpoint_data_model` (`fc4sc_coverpoint.hpp`)

`std::string name`	name of this coverpoint (inherited from `api_data_model` via `cvp_base_data_model`)
`uint64_t misses`	number of misses (miss = no bin hit when sampled) (inherited from `cvp_base_data_model`)
`cvp_option option`	see Table 6.10 (inherited from `coverpoint_base_data_model`)
`std::vector<bin_base_data_model*> bins_data`	regular bins (inherited from `coverpoint_base_data_model`)
`std::vector<bin_base_data_model*> illegal_bins_data`	illegal bins (inherited from `coverpoint_base_data_model`)
`std::vector<bin_base_data_model*> ignore_bins_data`	ignore bins (inherited from `coverpoint_base_data_model`)
`uint64_t size()`	number of regular bins
`std::string sample_expression_str`	sample expression as string
`std::string sample_condition_str`	sample condition as string
`std::string& get_sample_expression_str()`	sample expression as string
`std::string& get_sample_condition_str()`	sample condition as string

Table 6.10: Attributes of class `cvp_option` (`fc4sc_options.hpp`)

`uint/uint64_t weight = 1`	weight of the coverpoint inside its covergroup instance
`uint/uint32_t goal = 100`	coverage goal of the coverpoint (in percent)
`uint/uint64_t at_least = 1`	minimum number of hits of a bin to count this bin as covered
`uint/uint32_t auto_bin_max = 10`	for UCIS XML file, only
`bool detect_overlap = false`	for UCIS XML file, only
`std::string comment = ""`	for UCIS XML file, only

Table 6.11: Members of class `cross_data_model` (`fc4sc_cross.hpp`)

`std::string name`	name of this cross (inherited from `api_data_model` via `cvp_base_data_model`)
`uint64_t misses`	number of misses (miss = no bin hit when sampled) (inherited from `cvp_base_data_model`)
`cross_option option`	see Table 6.12 (inherited from `cross_base_data_model`)
`std::vector<cvp_base_data_model*> cross_cvps`	coverpoints that are used in this cross (Table 6.9) (inherited from `cross_base_data_model`)
`std::map<std::vector<size_t>, uint64_t> bins`	hit counter of the bins
`const std::map<std::vector<size_t>, uint64_t>& get_cross_bins() const`	hit counter of the bins, returns `bins`
`uint64_t size()`	number of cross bins

Member function `cross_data_model::get_cross_bins()` returns a map that associates cross bins with its hit counter. Each cross bin is specified by a vector with `cross_data_model::cross_cvps::size()` elements. If the ith element of the vector contains j then this entry refers to bin j of coverpoint i.

Table 6.12: Attributes of class `cross_option` (`fc4sc_options.hpp`)

`uint/uint64_t weight = 1`	weight of the cross inside its covergroup instance
`uint/uint32_t goal = 100`	coverage goal if the cross (in percent)
`uint/uint64_t at_least = 1`	minimum number of hits of a bin to count this bin as covered
`uint/uint32_t cross_num_print_missing = 0`	for UCIS XML file, only
`std::string comment = ""`	for UCIS XML file, only

6.8 Bin instances

The bins are actually managed by class `bin_data_model` which is derived from the abstract class `bin_base_data_model`. Since class `bin_data_model` is a template class with the data type of the bin as template parameter a typecast to this class requires the knowledge of this type. The class `bin_base_data_model` provides, therefore, virtual access functions that use type `int` instead. Please note that this will cause wrong values if the data type of the bin has a larger range than `int`.

Table 6.13: Members of class `bin_base_data_model` (`fc4sc_base.hpp`)

`std::string& get_name()`	name
`bin_t& get_bin_type()`	`default_`, `illegal_`, or `ignore_`
`std::vector<interval_t<int>> get_intervals_to_int() const`	intervals of this bin as `int`
`std::vector<uint64_t>& get_interval_hits()`	hit counter for each interval

Table 6.14: Members of class `bin_data_model<T>` (`fc4sc_bin.hpp`)

`std::string name`	name of the bin
`std::string& get_name()`	name of the bin
`bin_t bin_type`	`default_`, `illegal_`, or `ignore_`
`bin_t& get_bin_type()`	`default_`, `illegal_`, or `ignore_`
`std::vector<interval_t<T>> intervals`	intervals of this bin as `T`
`std::vector<interval_t<int>> get_intervals_to_int() const`	intervals of this bin as `int`
`std::vector<uint64_t> interval_hits`	hit counter for each interval
`std::vector<uint64_t>& get_interval_hits()`	hit counter for each interval

7 Exploring context by a visitor

FC4SC supports an access to the internal data using the visitor design pattern. A visitor class must be derived from `covVisitorBase` and implements at least its abstract public member functions (Table 7.1).

Table 7.1: Member functions of a visitor class

`void visit( scp_base_data_model& base)`
`void visit( cvg_base_data_model& base)`
`void visit(coverpoint_base_data_model& base)`
`void visit( cross_base_data_model& base)`
`void visit( bin_base_data_model& base)`

The visitor class can call the `accept_visitor()` member function of different data elements. This function

expects the visitor object as the only parameter. The `accept_visitor()` functions call the `visit()` function of its class.

Table 7.2: Classes that implement `accecpt_visitor()`

`scp_base_data_model::accept_visitor(visitor)`	calls `visitor.visit( scp_base_data_model&)`
`cvg_base_data_model::accept_visitor(visitor)`	calls `visitor.visit( cvg_base_data_model&)`
`coverpoint_data_model::accept_visitor(visitor)`	calls `visitor.visit(coverpoint_data_model&)`
`cross_data_model::accept_visitor(visitor)`	calls `visitor.visit( cross_data_model&)`
`bin_data_model::accept_visitor(visitor)`	calls `visitor.visit( bin_data_model&)`

The visitor class contains typically a member function to start the traversal through the data structure of a context. This function traverses all scopes (Listing 7.1). For each scope (`scp_base_data_model`), the `visit()` function is called.

```
my_visitor::main_traversal(cntxt)
{
  //...
  for (auto scope_inst : fc4sc::global::get_top_scopes(cntxt))
  {
    scope_inst->accept_visitor(*this); // *this is the visitor object
  }
  //...
}
```

Listing 7.1: Traversal of scopes

The `visit()` function of the scope traverses all covergroups and all subscopes (Listing 7.2).

```
void my_visitor::visit(scp_base_data_model& base)
{
  //...
  // direct traversal of all covergroup instances
  for (auto cvg_inst : base.cvg_insts)
  {
    cvg_inst.second.accept_visitor(*this);
  }
  //...
  // traversal of all covergroup instances type by type
  for (auto cvg_type : base.cvgs)
  {
    for (size_t i = 0; i < cvg_type.second.size(); i++)
    {
      cvg_type.second[i]->accept_visitor(*this);
    }
  }
  //...
  // traversal of all subscopes
  for (auto scope_inst_it : base.child_scp_insts)
  {
    scope_inst_it.second->accept_visitor(*this);
  }
}
```

Listing 7.2: Traversal of covergroups and sub scopes of a scope

The `visit()` function of the covergroup traverses all coverpoints (Listing 7.3). Depending on the nature of the coverpoint, the `visit()` function either for actual coverpoints or for crosses is called.

```
void my_visitor::visit(cvg_base_data_model& base)
{
  //...
  for (auto cvp : base.cvps)
  {
    cvp->accept_visitor(*this);
  }
  //...
}
```

Listing 7.3: Traversal of coverpoints of a covergroup

```
void my_visitor::visit(coverpoint_base_data_model& base)
{
  //...
  for (auto bin : base.bins_data)
  {
    bin->accept_visitor(*this);
  }
  // same for base.illegal_bins_data and base.ignore_bins_data (if needed)
  //...
}
```

Listing 7.4: Traversal of bins in a coverpoint

```
void my_visitor::visit(cross_base_data_model& base)
{
  //...
  for (auto cr : base.cross_cvps)
  {
    cr->accept_visitor(*this);
  }
  //...
}
```

Listing 7.5: Traversal of coverpoints of a cross

The class `xml_printer` for UCIS-export of coverage data is implemented as a visitor (Section 9.1).

8 Accessing data from objects

FC4SC provides functions to get some information directly from scope, covergroup, coverpoint, crosse, and bin. They allow the access to informations without traversing the complex data structure as describes in Section 6 or Section 7.

8.1 Accessing data from scopes

For each scope the member functions of Table 8.1 are defined. The constructors are described in Section 4.2.

Table 8.1: Public member functions of classes `scope` and `default_scope` (declarations derived from abstract class `scp_base`)

`scp_base_data_model*`	`get_scp_data()`	data model of the scope (Table 6.4)
`std::string&`	`name()`	name of the scope instance
`unsigned int&`	`file_id()`	ID of the file where this scope type is declared
`uint32_t&`	`line()`	line no. in file where this scope type is declared
`unsigned int&`	`inst_file_id()`	ID of the file where this scope instance is declared
`uint32_t&`	`inst_line()`	line no. in file where this scope instance is declared
`unsigned int&`	`instance_id()`	scope instance ID

8.2 Accessing data from covergroups

There are many functions to access different attributes of covergroups and its elements.

Table 8.2 and Table 8.3 summarize member functions of class `covergroup` that are inherited by each covergroup class. These functions are called for covergroup objects. While the values of Table 8.2 are specific to each covergroup class (type) are the values of Table 8.3 specific to each covergroup instance. "type" refers to the covergroup class.

Table 8.2: Member functions of `class covergroup` for type attributes (same value for all objects of a `covergroup` class)

`std::string&`	`type_name()`	name of the covergroup type
`cvg_type_option&`	`type_option()`	options of the type (Table 6.6)
`std::string&`	`scp_type_name()`	name of the scope type where this covergroup type belongs to
`unsigned int&`	`file_id()`	ID of the file where this type is declared
`uint32_t&`	`line()`	line no. in file where this type is declared

Table 8.3: Member functions of `class covergroup` for instance attributes

`std::string&`	`name()`	name of the covergroup instance
`cvg_option`	`option()`	options of the instance (Table 6.8)
`unsigned int&`	`inst_file_id()`	ID of the file where this covergroup object is instantiated
`uint32_t&`	`inst_line()`	line no. in file where this covergroup object is instantiated
`cvg_base_data_model*`	`get_cvg_data()`	pointer to the internal data structure of the covergroup (Table 6.7)

8.3 Accessing data from coverpoints

For each coverpoint the functions of Table 8.4 are defined. The constructors are described in Section 4.4.1. The functions to control the sampling are described in Section 5. The functions to access the coverage result are described in Section 9.2.

Table 8.4: Overview: member functions of `class coverpoint`

`std::string&`	`name()`	name of the coverpoint
`cvp_option&`	`option()`	option of the coverpoint (Table 6.10)
`uint64_t`	`get_weight()`	weight of the coverpoint (Table 6.10)
`std::vector<bin_base*>`	`get_bins_base()`	vector of regular bins (Section 8.4)
`std::vector<bin_base*>`	`get_ignore_bins_base()`	vector of ignore bins (Section 8.4)
`std::vector<bin_base*>`	`get_illegal_bins_base()`	vector of illegal bins (Section 8.4)
`cvp_base_data_model*`	`get_data()`	pointer to the data model (Section 6.7)
`std::string&`	`get_sample_expression_str()`	sample expression
`std::string&`	`get_sample_condition_str()`	sample condition
`uint64_t`	`size()`	number of bins

8.4 Accessing data from bins

In many cases the bin objects cannot been accessed directly because they are only created for construction of coverpoints. (Exceptions see at end of Section 4.4.3.) There are member functions of coverpoints that allow access to class `bin_base` which is an abstract base class of class `bin`.

Table 8.5: Overview: member functions of `class bin`, derived from `bin_base`

`std::string&`	`name()`	name of the bin
`bin_t&`	`bin_type()`	type of the bin (Table 6.13)
`std::vector<uint64_t>&`	`interval_hits()`	number of hits for each interval of the bin
`std::vector<interval_t<int>>`	`get_intervals_to_int()`	intervals casted to `int`
`uint64_t`	`get_hitcount()`	Retrieves the number of hits of the bin (sum of all intervals)

8.5 Accessing data from crosses

Table 8.6: Overview: member functions of `class cross`

`std::string&`	`name()`	name of the cross
`cross_option&`	`option()`	cross options
`std::vector<cvp_base*>`	`cvps_vec`	coverpoints of the cross
`const std::vector<cvp_base*>&`	`get_cross_coverpoints()`	coverpoints of the cross
`cvp_base_data_model*`	`get_data()`	data model of the cross (Section 6.7)
`uint64_t`	`size() const`	number of cross bins
`const std::map<std::vector<size_t>, uint64_t>&`	`get_cross_bins() const`	same result as `get_cross_bins()` of `cross_base_data_model` (Table 6.11)

Class `cvp_base` is an abstract base class of `coverpoint`. The pointers to `cvp_base` in `cross` point to `coverpoint` objects and can be downcasted. Without downcast, a limited set of functions can be used, see Table 8.7.

Table 8.7: Overview: member functions of `cvp_base`. For explanations see Table 8.4.

`uint64_t`	`get_weight()`
`uint64_t`	`size()`
`uint64_t`	`get_misses()`
`cvp_base_data_model*`	`get_data()`

9 Accessing coverage data

9.1 Export coverage data

The Unified Coverage Interoperability Standard (UCIS) defines interfaces for working with coverage data [Acc26b][Acc12]. For data exchange an XML format is specified. The coverage data of all covergroup objects can be written to an UCIS compliant file. When calling `xml_printer::coverage_save()`, either a `std::ofstream` or a file name can be used to specify the XML file.

```
void f()
{
  xml_printer::coverage_save(<file_name>);
  xml_printer::coverage_save(<file_name>, <context>);
  xml_printer::coverage_save(<ofstream>);
  xml_printer::coverage_save(<ofstream>, <context>);
  // e.g.
  xml_printer::coverage_save("coverage.xml"); // global/default context
  xml_printer::coverage_save("coverage.xml", cntxt);
  std::ofstream stream;
  // open file stream
  xml_printer::coverage_save(stream);
}
```

Listing 9.1: Export of coverage data to UCIS XML file

In an UVM testbench it is common to export coverage in the report phase. Since this action is global for the whole testbench a central testbench object must carry out this task, e.g. the top-level scoreboard.

If the coverage is of interest even in case the testbench found a deviation then this function must be called also immediatly before the testbench is stopped e.g. before the `UVM_FATAL()` call.

The `xml_printer` is a visitor class as described in Section 7.

9.2 Accessing detailed coverage data

There are several function to get detailed coverage information similar to that stored in the UCIS XML file.

The coverage related members of `class coverpoint` are summarized in Table 9.1.

Table 9.1: Member of `class coverpoint` for getting coverage information

`double get_inst_coverage()`	coverage in percent
`double get_inst_coverage(int& covered, int& total)`	coverage in percent, `covered` = $res(cp)$, `total` = $num_regular_bins(cp)$ = `size()`
`uint64_t get_weight()`	weight of the coverpoint
`uint64_t get_misses()`	the number of misses, i.e. number of samplings where no regular bin has been hit
`uint64_t get_bin_hit_count(uint32_t bin_index)`	number of hits of bin `bin_index` i.e. $hitcount(bin(bin_index))$ Throws an error if the `bin_index` is out of bounds.

The following formulas describe the coverage of a coverpoint cp. Please note that $at_least(cp)$, $weight(cp)$, and $goal(cp)$ are parameters (options) of coverpoint cp. The parameters are defined in the options of the coverpoint (Table 6.10).

$$hitcount(bin) = \sum_{interval \in intervals(bin)} hitcount(interval) \tag{2}$$

$$hit(cp, bin) = \begin{cases} 1 & \text{if } hitcount(bin) \geq at_least(cp) \\ 0 & \text{otherwise} \end{cases} \tag{3}$$

$$res(cp) = \sum_{bin \in regular_bin(cp)} hit(cp, bin) \tag{4}$$

$$real(cp) = \frac{res(cp) * 100}{num_regular_bins(cp)} \tag{5}$$

$$coverage(cp) = \begin{cases} 100 & \text{if } num_regular_bins(cp) = 0 \wedge weight(cp) = 0 \\ 0 & \text{if } num_regular_bins(cp) = 0 \wedge weight(cp) \neq 0 \\ 100 & \text{if } real(cp) \geq goal(cp) \\ real & \text{otherwise} \end{cases} \tag{6}$$

The coverage related members of `class cross` are summarized in Table 9.2.

Table 9.2: Members of `class cross` for getting coverage information

`double get_inst_coverage() const`	coverage in percent
`double get_inst_coverage(int& covered, int& total)`	coverage in percent, `covered` = $covered(cr)$, `total` = $num_bins(cr)$ = `size()`
`uint64_t get_weight()`	weight of the cross
`uint64_t get_misses()`	number of misses.

The following formulas describe the coverage of a cross cr. Please note that $at_least(cr)$, $weight(cr)$, and $goal(cr)$ are parameters (options) of cross cr. The parameters are defined in the options of the cross (Table 6.12).

$$hit(cr, cr_bin) = \begin{cases} 1 & \text{if } hitcount(cr_bin) \geq at_least(cr) \\ 0 & \text{otherwise} \end{cases} \tag{7}$$

$$covered(cr) = \sum_{cr_bin \in cr} hit(cr, cr_bin) \tag{8}$$

$$real(cr) = \frac{covered(cr) * 100}{num_bins(cr)} \tag{9}$$

$$coverage(cr) = \begin{cases} 100 & \text{if } num_bins(cr) = 0 \wedge weight(cr) = 0 \\ 0 & \text{if } num_bins(cr) = 0 \wedge weight(cr) \neq 0 \\ 100 & \text{if } real(cr) \geq goal(cr) \\ real(cr) & \text{otherwise} \end{cases} \tag{10}$$

The coverage related members of `class covergroup` are summarized in Table 9.3. If the context (`cvg_cntxt`) is not specified (default to `nullptr`) then the context is used to which the scope of the covergroup belongs to.

Table 9.3: Members of `class covergroup` for getting coverage information

`double get_inst_coverage()`	coverage of the covergroup object in percent
`double get_inst_coverage(int& bins_covered, int& total)`	coverage of the covergroup object in percent, `bins_covered` = $bins_covered(cgi)$, `total` = $total(cgi)$
`double get_coverage(global* cvg_cntxt)`	coverage of the covergroup type, uses `get_coverage()` of the context with `type` = name of this covergroup type (Table 9.4)
`double get_coverage(int& bins_covered, int& total, global* cvg_cntxt)`	coverage of the covergroup type, uses `get_coverage()` of the context with `type` = name of this covergroup type, `bins_covered` = $bins_covered(cgt)$, `total` = $total(cgt)$ (Table 9.4)

Assuming that a covergroup instance cgi contains the set CP of coverpoints and the set CR of crosses then the following formulas describe the coverage of an instance cgi of a covergroup. If there is no coverpoint and no cross in the covergroup then $res(cgi) = 0$. Please note that $weight(cp)$ is a parameter (option) of coverpoint cp (Table 6.10), $weight(cr)$ is a parameter of cross cr (Table 6.12), and $weight(cgi)$ and $goal(cgi)$ are parameters of the covergroup instance cgi (Table 6.8).

$$bins_covered(cgi) = \sum_{cp \in CP} res(cp) + \sum_{cr \in CR} covered(cr) \tag{11}$$

$$total(cgi) = \sum_{cp \in CP} num_regular_bins(cp) + \sum_{cr \in CR} num_bins(cr) \tag{12}$$

$$res(cgi) = \sum_{cp \in CR} coverage(cp) \cdot weight(cp) + \sum_{cr \in CR} coverage(cr) \cdot weight(cr) \tag{13}$$

$$weights(cgi) = \sum_{cp \in CP} weight(cp) + \sum_{cr \in CR} weight(cr) \tag{14}$$

$$real(cgi) = \frac{res(cgi)}{weights(cgi)} \tag{15}$$

$$coverage(cgi) = \begin{cases} 100 & \text{if } ((weights(cgi) = 0) \vee (res(cgi) = 0)) \wedge weight(cgi) = 0 \\ 0 & \text{if } ((weights(cgi) = 0) \vee (res(cgi) = 0)) \wedge weight(cgi) \neq 0 \\ 100 & \text{if } real(cgi) \geq goal(cgi) \\ real(cgi) & \text{otherwise} \end{cases} \tag{16}$$

Assuming that a covergroup type cgt has the set CGI of covergroup instances then the following formulas describe the coverage of a covergroup type cgt. If there is no covergroup instance for the covergroup type then $res(cgt) = 0$. Please note that $weight(cgi)$ is a parameter (option) of covergroup instance cgi (Table 6.8) and $weight(cgt)$ as well as $goal(cgt)$ are parameters of covergroup type cgt (Table 6.6).

$$bins_covered(cgt) = \sum_{cgi \in CGI} bins_covered(cgi) \tag{17}$$

$$total(cgt) = \sum_{cgi \in CGI} total(cgi) \tag{18}$$

$$res(cgt) = \sum_{cgi \in CGI} res(cgi) \cdot weight(cgi) \tag{19}$$

$$weights(cgt) = \sum_{cgi \in CGI} weight(cgi) \tag{20}$$

$$real(cgt) = \frac{res(cgt)}{weights(cgt)} \tag{21}$$

$$coverage(cgt) = \begin{cases} 100 & \text{if } ((weights(cgt) = 0) \vee (res(cgt) = 0)) \wedge weight(cgt) = 0 \\ 0 & \text{if } ((weights(cgt) = 0) \vee (res(cgt) = 0)) \wedge weight(cgt) \neq 0 \\ 100 & \text{if } real(cgt) \geq goal(cgt) \\ real(cgt) & \text{otherwise} \end{cases} \tag{22}$$

The coverage related members of contexts are summarized in Table 9.4.

Table 9.4: Members of context for getting coverage information

`double global::get_coverage(const std::string& scp_type, const std::string& type, global* cvg_cntxt)`	coverage $coverage(cgt)$ of all covergroup instances of a given `type` (cgt) of a scope type `scp_type` in percent
`double global::get_coverage(const std::string& scp_type, const std::string& type, int& hit_bins, int& total_bins), global* cvg_cntxt)`	coverage $coverage(cgt)$ of all covergroup instances of a given `type` (cgt) of a scope type `scp_type` in percent, `hit_bins` = $bins_covered(cgt)$, `total_bins` = $total(cgt)$
`double global::get_coverage(global* cvg_cntxt)`	coverage $coverage$ of all covergroup classes of a context

Assuming that the context contains the set SCP of scopes and each scope $scp \in SCP$ contains the set $CGT(scp)$ of covergroup types then the following formulas describe the coverage of the context. Please note that $weight(cgt)$ is a parameter (option) of covergroup type cgt (Table 6.6).

$$res = \sum_{scp \in SCP} \sum_{cgt \in CGT(scp)} coverage(cgt) \cdot weight(cgt) \tag{23}$$

$$weights = \sum_{scp \in SCP} \sum_{cgt \in CGT(scp)} weight(cgt) \tag{24}$$

$$coverage = \begin{cases} 100 & \text{if } SCP = \emptyset \\ 0 & \text{if } SCP \neq \emptyset \wedge weights = 0 \\ \frac{res}{weights} & \text{otherwise} \end{cases} \tag{25}$$

10 Additional tools

The FC4SC version of Amiq [AMI18] provides some additional tools that work on the UCIS XML files created by FC4SC. These tools do not run on the files produced by the FC4SC version provided by Accellera. There are patches available for these tools at [Sch26] that allow usage with the UCIS XML files created by the Accellera version.

10.1 Graphical representation

The HTML file `contrib/tools/gui/index.html` is provided to visualize coverage results in a browser. When open this file, one can see on the top a button to load an UCIS XML file (Figure 10.1). The content of the file is then represented by tables. Each covergroup instance is represented by a table (Figure 10.2). The table shows the hit counter for each bin, the coverage for each coverpoint and cross and the coverage for the whole covergroup instance. Please note, that the coverage values are computed inside the tool and may differ from the values provided by FC4SC because the coverage values are not contained in the UCIS XML file.

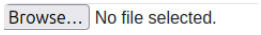


Figure 10.1: File open button

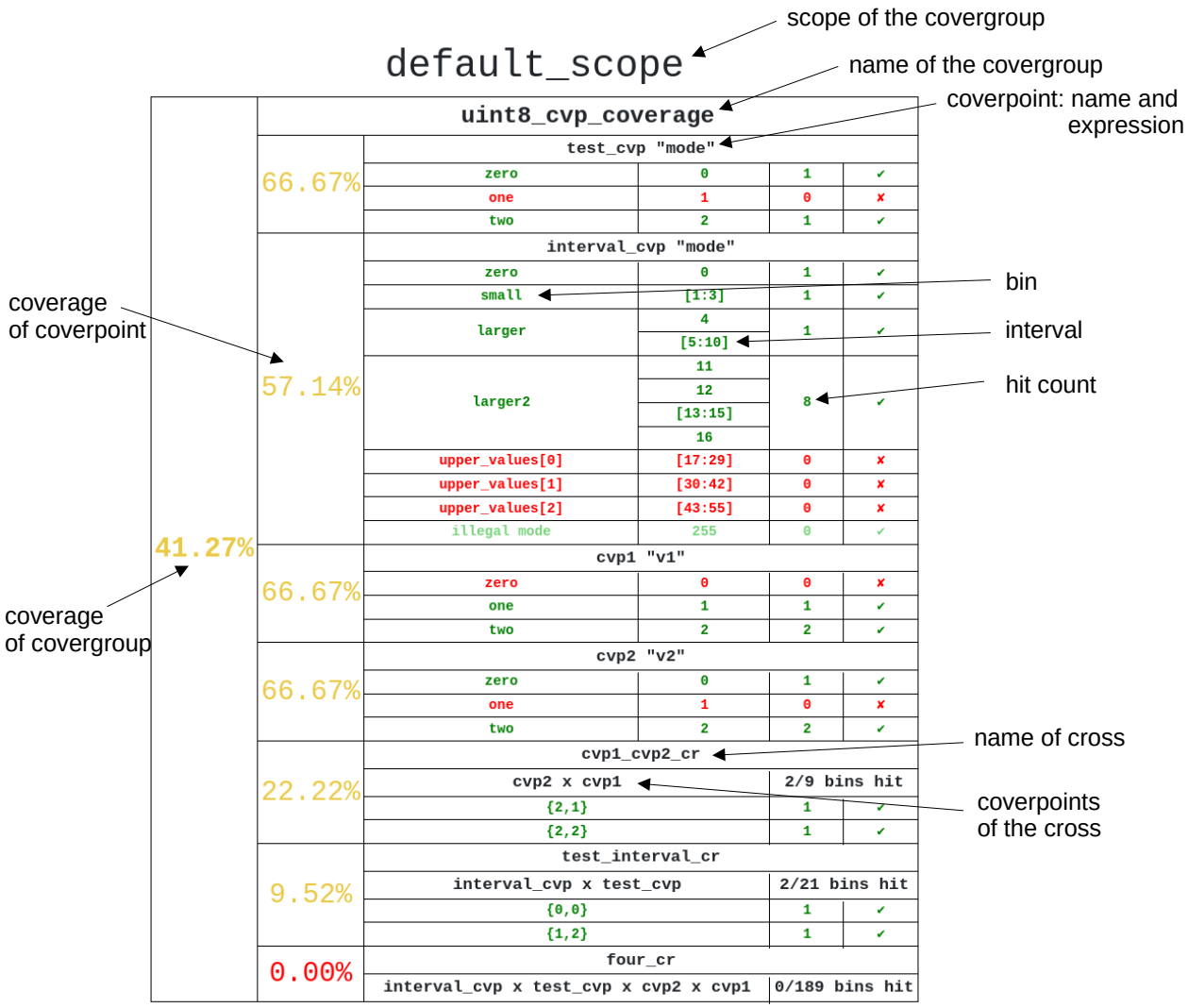


Figure 10.2: Table with coverage results

10.2 Merge

The python script `contrib/tools/coverage_merge/merge.py` allows to merge the content of two or more UCIS XML files into one UCIS XML file. The hit counters for each interval are summed up. The merge assumes that the XML files come from the same scope and covergroup structure. It is not supported to merge XML files from different structures.

There are different ways to use this merge script. One way is to specifie the result file explicitely by option `--merge_to_db` (Listing 10.1). The result file can be one of the source files. In this case the content of this source file is overwritten. This feature can be used to accumulate coverage data in one file.

```
python3 merge.py --merge_to_db=<result-file> <source-file-1> <source-file-2> ...
```

Listing 10.1: Merge of several XML files into one merged XML file

If this option `--merge_to_db` is not specified then all XML files of a specified directory are merged (Listing 10.2). The directory is the first parameter and the name of the result file is the second parameter of the script. If the result file name is not specified then `coverage_merged_db.xml` is used instead. If the directory name is not specified then the current working directory is used. If the result file is one of the XML files found in the source directory then this file is not used as source file.

```
python3 merge.py <source-dir> <result-file>
python3 merge.py <source-dir>                # result-file = coverage_merged_db.xml
python3 merge.py  # source-dir is current dir, result-file = coverage_merged_db.xml
```

Listing 10.2: Merge of all XML files of a directory

References

[Acc12] Accellera Unified Coverage Interoperability Standard (UCIS) Working Group. (2012) UCIS 1.0. [Online]. Available: https://www.accellera.org/downloads/standards/ucis

[Acc19] Accellera SystemC Verification Working Group. (2019) FC4SC git repository. [Online]. Available: https://github.com/accellera-official/fc4sc

[Acc26a] ——. (2026) Accellera systemc verification working group. [Online]. Available: https://www.accellera.net/activities/working-groups/systemc-verification

[Acc26b] Accellera Unified Coverage Interoperability Standard (UCIS) Working Group. (2026) Accellera unified coverage interoperability standard (UCIS) working group. [Online]. Available: https://www.accellera.org/activities/working-groups/ucis

[AMI18] AMIQ Consulting. (2018) Git repository of amiq. This repo was archived on Oct 7, 2026. It is now read-only. [Online]. Available: https://github.com/amiq-consulting/fc4sc

[Sch26] J. Schönherr. (2026) Usefull things for UVM-SystemC. [Online]. Available: https://github.com/jens-schoenherr/uvm-systemc-htwd-contrib